https:leanpub.com/daa

渐近分析

asymptotic analysis

当n→∞时，函数f(n)的极限行为

Youtube频道：hwdong

https://github.com/hwdong-net/DAA

“渐近分析”（asymptotic analysis）

- 指的是分析算法的性能（主要是时间复杂度和空间复杂度）在输入规模趋近于无穷大的情况下的表现。
- 该术语源于数学中的“渐近线”（asymptote），描述函数在趋于无穷时的极限行为。

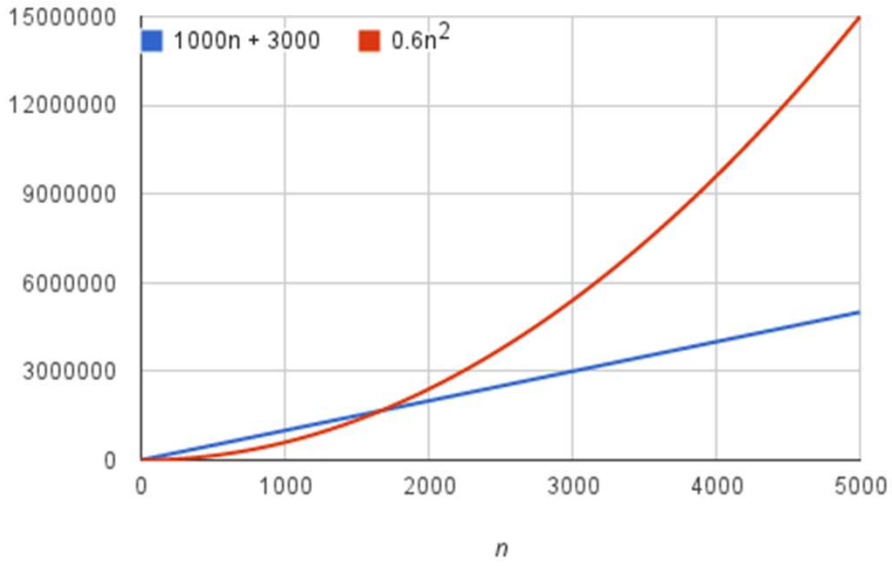
渐近分析

- 设$f(n) = n^2 + 3n$，当n变得非常大时，与n^2 相比，3n变得微不足道，因此，当$n \to \infty$时，$f(n)$ 和n^2 渐进相等。即 $f(n) \sim n^2$。

- 当n→∞时，

  1000n+3000和0.6n²

  比较可以忽略不计。



渐近分析

- 通过去掉不那么重要的项和常数系数，我们可以专注于算法运行时间的重要部分，即增长率，而不会陷入使我们的理解更加复杂的细节中。
- 如$0.01n^2+700n$和n^2是同一个数量级，它们具有同样的增长率（同介无穷大量）。因此，

  $$0.01n^2+700n = O(n^2)$$

- 当去掉常数系数和不太重要的项时，使用**渐近符号**(如O)表示增长的数量级。

渐近符号

- 上界O、下界Ω、紧界Θ、非紧上界o、非紧下界ω。

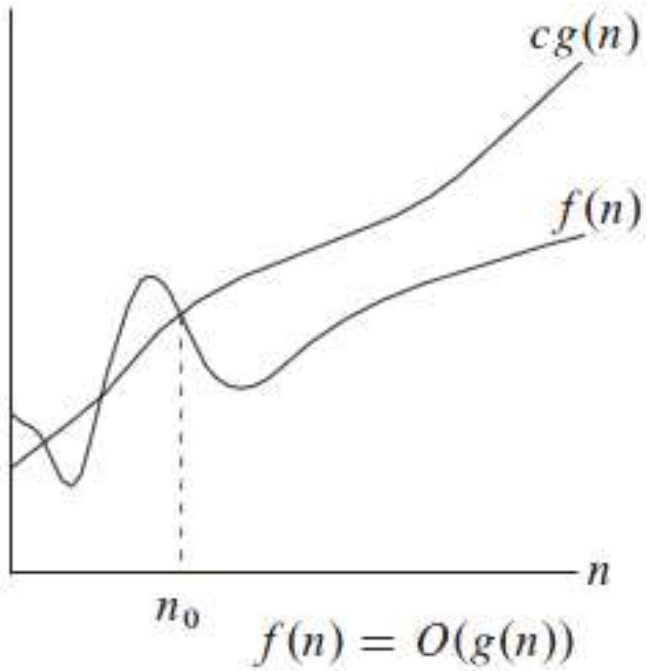
上界O

- 若存在正数c和n_0，使得对所有$n \geq n_0$有：

$$0 \leq f(n) \leq cg(n),$$

则称f(n)的渐进上界是g(n)，记作

$$f(n)=O(g(n))$$



最坏情况下，f(n)不会比g(n)增长快

上界O

- 表示算法的**最坏情况复杂度**，即 **最多增长的速度**。
- 如$f(n)= O(n^2)$，这意味着：

  即使在最坏情况下，算法的运行时间最多是 n^2 的量级。

  换句话说，算法的执行时间不会比 n^2 级别的增长更快。

上界O

- 如$f(n) = n^2+2n$,则:

$f(n)=O(n^2)$,可取$c = 2, n_0 = 2$

$f(n)=O(n^3)$,可取$c = 1, n_0 = 2$

$f(n)=O(0.1n^3)$,可取$c = 10, n_0 = 2$

- 如果$f(n)=O(g(n))$，说明$g(n)$的阶等于或高于$f(n)$.

上界的阶越低，则评估就越精确，就越有价值。

求证：所有度为k的多项式的上界是$O(n^k)$。

证明：设$T(n) = a_k n^k + \ldots + a_1 n + a_0$ where $a_k \neq 0$.

Let $n_0 = 1$ and let $a^* = \max_i |a_i|$

$$
\begin{aligned}
T(n) &= a_k n^k + \ldots + a_1 n + a_0 \\
&\leq a^* n^k + \ldots + a^* n + a^* \\
&\leq a^* n^k + \ldots + a^* n^k + a^* n^k \\
&= (k+1) a^* \cdot n^k
\end{aligned}
$$

Let $c = (k+1)a^*$ which is a constant

练习

- 请证明：对函数$f(x) = 2x^2 - 5x + 1$, $f(x) = O(x^2)$
- 请证明：$For\ any\ k \geq 1,\ n^k\ is\ not\ O(n^{k-1})$.

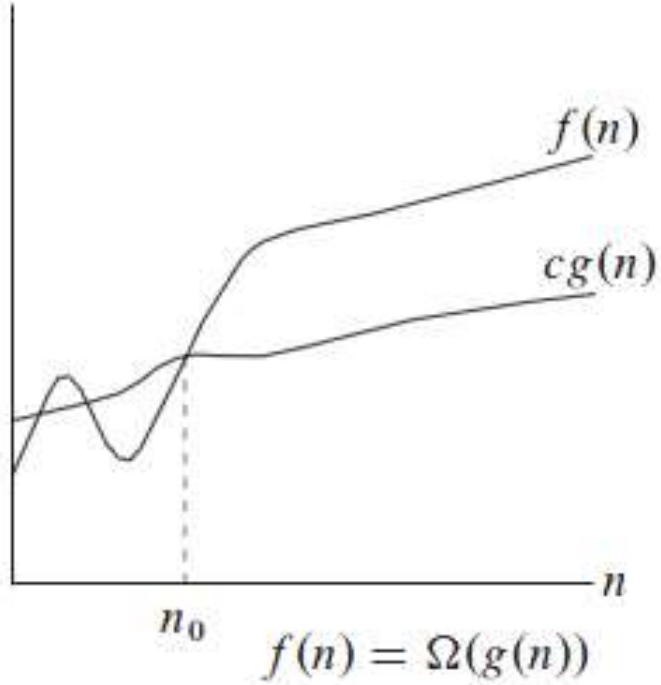
下界Ω

- 若存在正数c和n_0，使得对所有$n \geq n_0$有：

$$0 \leq cg(n) \leq f(n),$$

则称f(n)的渐进下界是g(n)，记作

$$f(n) = \Omega(g(n))$$



最优情况下，f(n)不会比g(n)增长慢

下界Ω

- 表示算法的 **最优情况复杂度**，即 **至少增长的速度**。
- 如$f(n)=\Omega(n^2)$，这意味着：

  无论输入情况多么有利，运行时间至少是 n^2 的量级。

  换句话说，最优情况下的运行时间也不会低于 n^2 。

下界Ω

- 如$f(n) = n^2+2n$,则:

$f(n)= \Omega(n^2)$,可取$c = 1, n_0 = 1$

$f(n)= \Omega(100n)$,可取$c = 0.01, n_0 = 1$

$f(n)= \Omega(10n^2+5n)$,可取$c = 0.1, n_0 = 1$

- 如果$f(n)= \Omega(g(n))$，说明$g(n)$的介等于或低于$f(n)$.

下界的阶越高，则评估就越精确，就越有价值。

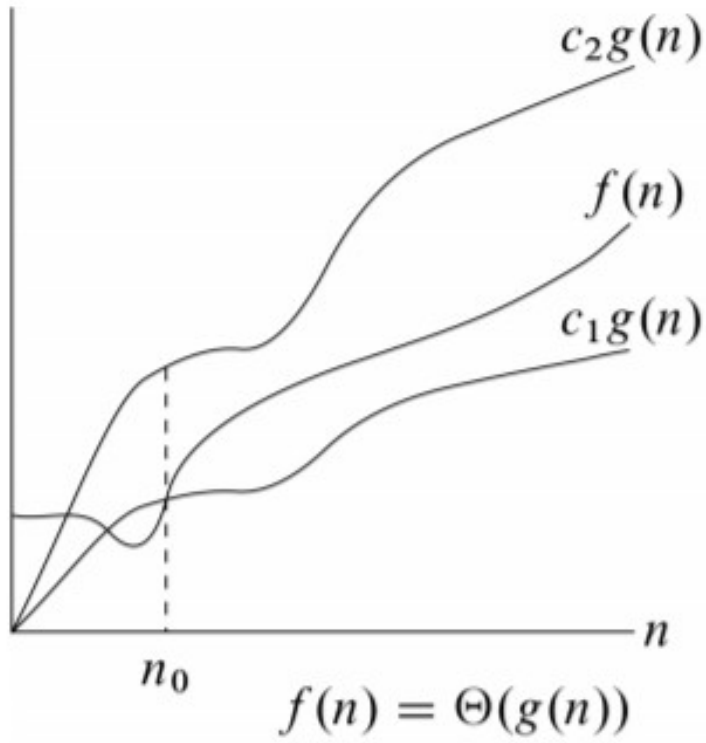
紧界Θ

- 若正数c_1,c_2和n_0使得对所有$n \geq n_0$有：

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \},$$

则称f(n)的紧界是g(n)，记作

$$f(n) = \Theta(g(n))$$



无论最优还是最坏， f(n)的增长速率都在 g(n)级别

紧界Θ

- 如果f(n)的增长率既不会超过g(n)，又不会低于g(n)，那么g(n)就是f(n)的精确的增长率。表示为：

  $f(n) = \Omega(g(n))$

- 如$\Omega(n^2)$代表的是“紧界”，意味着算法的运行时间既不会比 n^2更快，也不会比 n^2更慢。
- **紧界**表示算法最优和最坏情况下的增长率是一样的。

紧界Θ

- 如$f(n) = n^2+2n$, $g(n) = n^2$,则$f(n)= \Theta(n^2)$
- 如果$f(n)= \Theta(g(n))$，说明f(n)和g(n)是同阶无穷大量。
- 定理： $\lim_{n\to\infty} f(n)/g(n) =c$ $(c>0)$, 则$f(n)= \Theta(g(n))$

如：

$\lim_{n\to\infty} (n^2/3-4n)/(n^2) = 1/3$，因此$(n^2/3-4n) = \Theta(n^2)$

练习

- $f(n) = 0.1n^2+3n-7$和n,n^2,n^3的关系

非紧上界o

- 若对**任何**正数c>0，存在正数n_0 >0使得对所有n≥ n_0有：

  $0 \le f(n)<cg(n)$ }

则称f(n)的非紧上界是g(n)，记作

$$f(n)= o(g(n))$$

- 等价于 $f(n) / g(n) \to 0$， as $n\to\infty$。

如： $\lim_{n\to\infty}(5n^2+9n)/(n^3) = 0$，因此$(5n^2+9n) = o(n^3)$

非紧上界o

- $\log n = o(n^c)$
- $n^c = o(a^n)$

非紧下界ω

- 若对**任何**正数c>0，存在正数n_0>0使得对所有n≥ n_0有：

$$0 \le cg(n) < f(n) \}$$

则称f(n)的非紧下界是g(n)，记作

$$f(n)= \omega (g(n))$$

$f(n) = n^2$

$g(n) =10n$

- 等价于 $f(n) / g(n) \to \infty$， as $n \to \infty$。

性质：

(1) 传递性

- 如果$f=O(g), g=O(h)$，则$f=O(h)$
- 如果$f=\Omega(g), g=\Omega(h)$，则$f=\Omega(h)$
- 如果$f=\Theta(g), g=\Theta(h)$，则$f=\Theta(h)$
- 如果$f=o(g), g=o(h)$，则$f=o(h)$
- 如果$f=\omega(g), g=\omega(h)$，则$f=\omega(h)$

$f \le g, g \le h$, 则$f \le h$

$f \ge g, g \ge h$, 则$f \ge h$

（2）反身性：

- $f(n)= \Theta(f(n))$；
- $f(n)= O(f(n))$；
- $f(n)= \Omega(f(n))$.

（3）对称性：

- $f(n)= \Theta(g(n)) \Leftrightarrow g(n)= \Theta(f(n))$

（4）互对称性：

- $f(n)= O(g(n)) \Leftrightarrow g(n)= \Omega(f(n))$；
- $f(n)= o(g(n)) \Leftrightarrow g(n)= \omega(f(n))$；

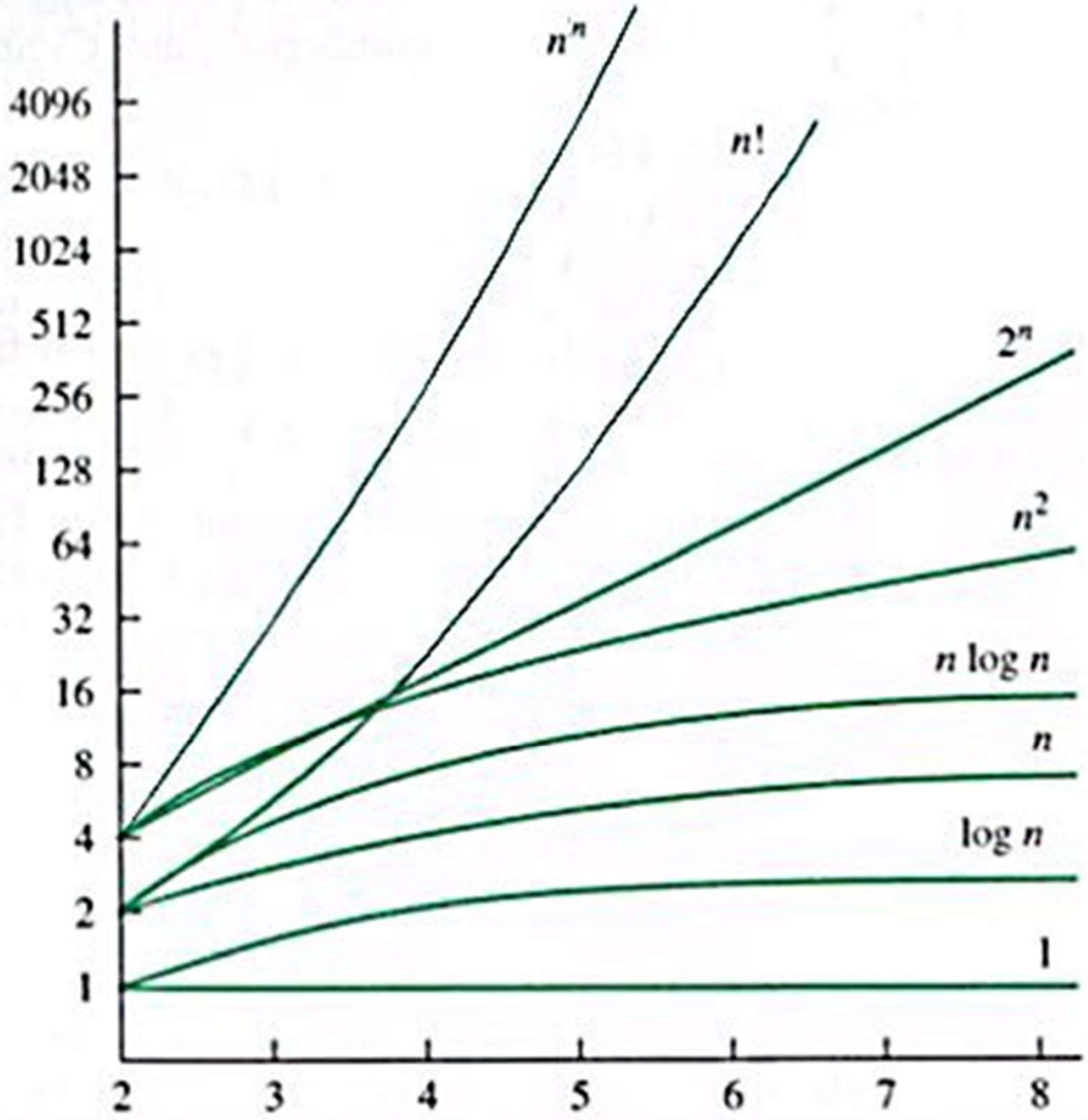
算法分析中常见的复杂性函数

Function	Name	
c	Constant	常数
$\log N$	Logarithmic	对数
$\log^2 N$	Log-squared	
N	Linear	线性
$N \log N$	$N \log N$	
N^2	Quadratic	二次
N^3	Cubic	三次
2^N	Exponential	指数

多项式

算法分析中常见的复杂性函数

Function	Name		
c	Constant	常数	O(1)
$\log N$	Logarithmic	对数	O(logn)
$\log^2 N$	Log-squared		
N	Linear	线性	O(n)
$N \log N$	$N \log N$		O(nlogn)
N^2	Quadratic	二次	$O(n^2)$
N^3	Cubic	三次	$O(n^3)$
2^N	Exponential	指数	$O(2^n)$


n^n
$n!$
2^n
n^2
$n \log n$
n
$\log n$
1
4096
2048
1024
512
256
128
64
32
16
8
4
2
1
2
3
4
5
6
7
8

- 哪种增长最能体现这些函数的特点？

	Constant	Linear	Polynomial	Exponential
$(3/2)^n$	○	○	○	○
1	○	○	○	○
$(3/2)n$	○	○	○	○
$2n^3$	○	○	○	○
2^n	○	○	○	○
$3n^2$	○	○	○	○
1000	○	○	○	○
$3n$	○	○	○	○

常见的时间复杂度

例子

Youtube频道： hwdong

常数时间: O(1)

- O(1) 表示算法的执行时间不会随着输入规模 n 的变化而变化，它始终是一个常数级别的操作。
- 换句话说，无论输入数据有多大，算法的执行时间都保持不变，不受 n 影响。

常数时间: O(1)

- 示例 1：数组的索引访问

```
int fun(int arr[], int i, int n){
    int x = arr[i];  // 直接访问索引 i 处的元素
    …
}
```

对arr[i]，无论i是0，1，3，5，都只需要 一次内存读取。

常数时间: O(1)

- 示例 2：哈希表（Hash Table）查找

```
unordered_map<int, string> myMap;
myMap[10] = "Apple";
myMap[20] = "Banana";

// 直接查询 key = 10 的值
string value = myMap[10];
```

无论哈希表存储 10 个元素还是 1000000 个元素，查询某个 key 的时间基本不变，即 O(1)

常数时间: O(1)

- 示例 3：栈的 push 和 pop 操作

```
stack<int> s;
s.push(10);
s.push(20);
s.pop();
```

不管栈里有 1 个元素还是 100 万个元素，push/pop 操作的时间始终不变。

常数时间: O(1)

- 只要操作**不会随着输入规模增大而变慢**，它的时间复杂度就是 **O(1)**。

操作	是否依赖n	时间复杂度
数组索引访问 arr[i]	否	**O(1)**
哈希表查找 map[key]	否(最优情况)	**O(1)**
栈的 push/pop	否	**O(1)**

线性时间linear time : O(n)

- 运行时间最多是输入规模的常数倍，如cn。
- 如：求一组数的(a_1,a_2,…a_n)最大值。

```
max = a_1
for i=2 to n:
    if a_i >max:
        max = a_i
return max
```

线性时间linear time : O(n)

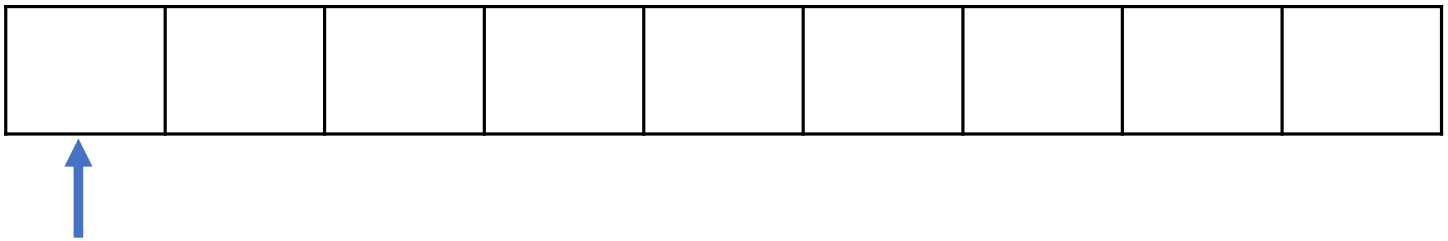
- 2个有序数组的合并(merge)：

  $a = (a_1, a_2, \ldots a_m)$, $b = (b_1, b_2, \ldots b_n)$

如： $a = (2,5,8,9)$, $b = (3,4,6,10,13)$

i j

c =



k

线性时间linear time : O(n)

- 2个有序数组的合并(merge)：

  $a = (a_1, a_2, \ldots a_m)$, $b = (b_1, b_2, \ldots b_n)$

如： $a = (2,5,8,9)$, $b = (3,4,6,10,13)$

i j

c =

2								

k

线性时间linear time : O(n)

- 2个有序数组的合并(merge)：

  $a = (a_1, a_2, \ldots a_m)$, $b = (b_1, b_2, \ldots b_n)$

如： $a = (2,5,8,9)$, $b = (3,4,6,10,13)$

i j

c =

2	3							

k

线性时间linear time : O(n)

- 2个有序数组的合并(merge)：

  $a = (a_1, a_2, \ldots a_m)$, $b = (b_1, b_2, \ldots b_n)$

如： $a = (2,5,8,9)$, $b = (3,4,6,10,13)$

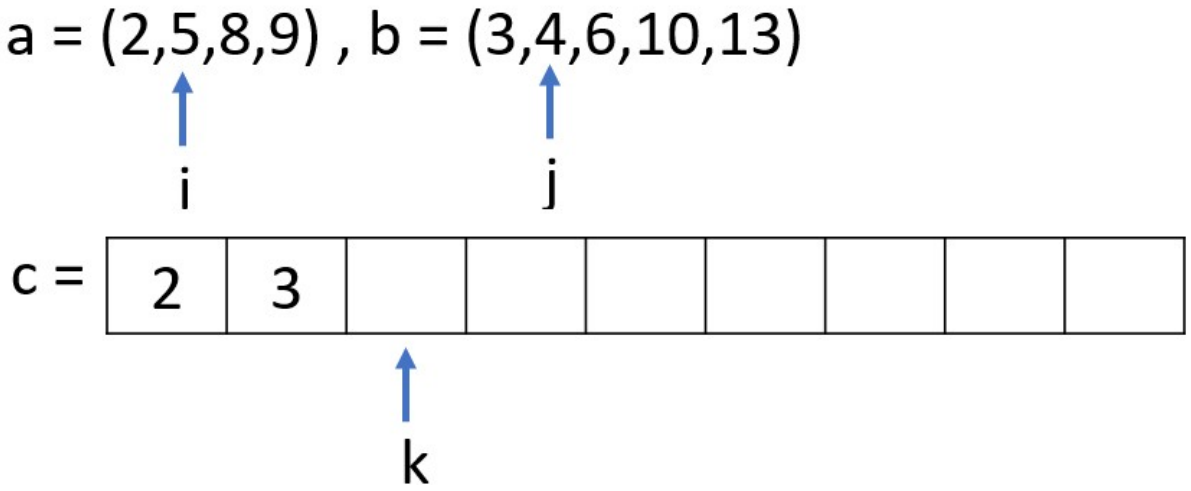
i j

c =

2	3	4						

k

merge(a,b)



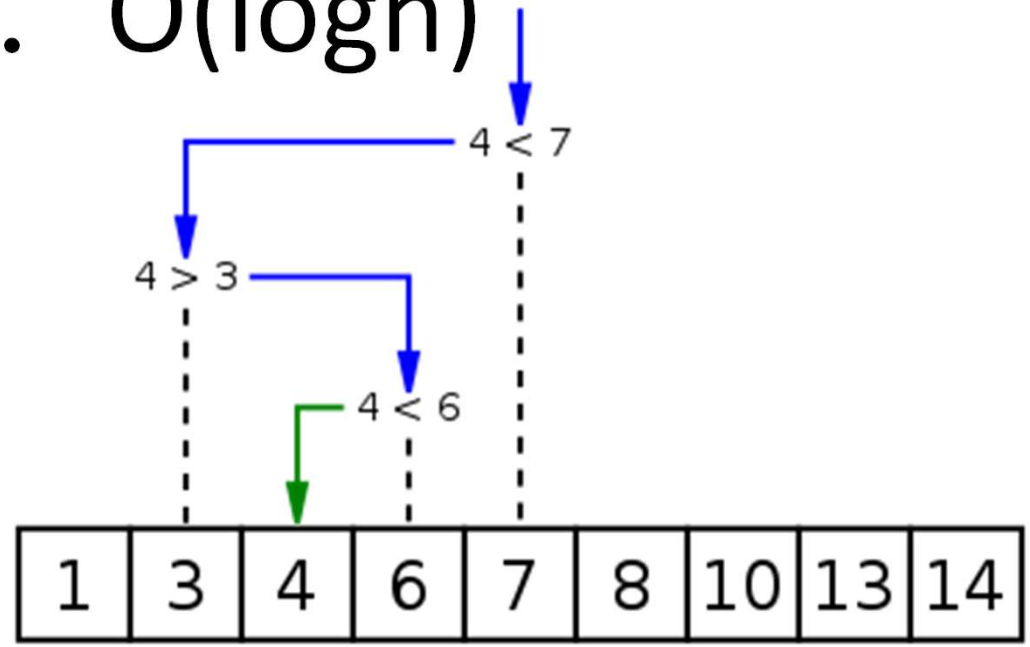

```
i=1,j=1
while i ≤m and j ≤n:
    if a_i ≤ b_j :
        c_k = a_i ,i++,k++
    else:
        c_k = b_j ,j++,k++
while i ≤m:
    c_k = a_i ,i++,k++
while j ≤n:
    c_k = b_j ,j++,k++
```

基本操作

O(m+n)

对数时间：O(logn)

- 有序数组的二分查找

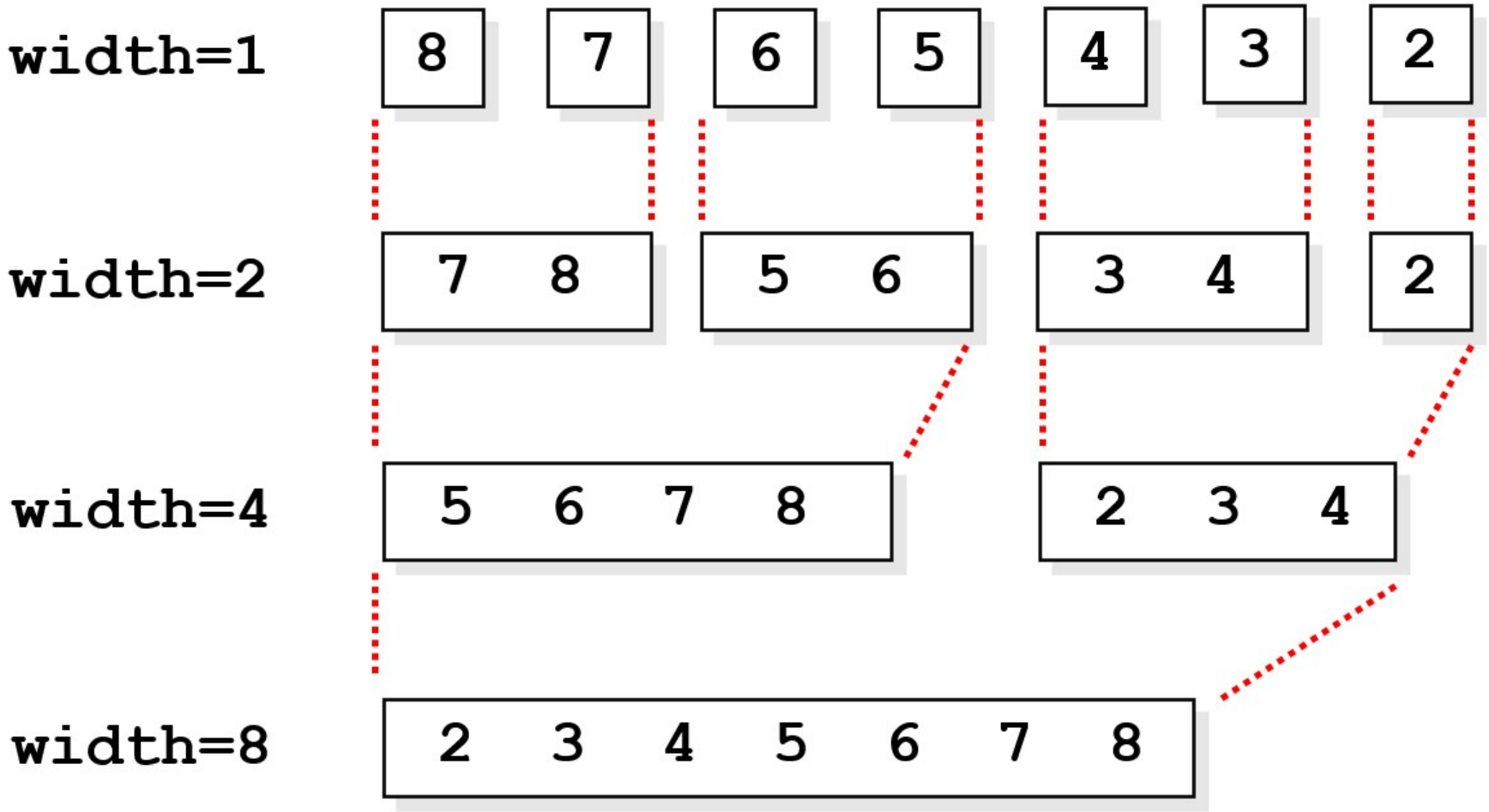


复杂度	
平均时间复杂度	$O(\log n)$
最坏时间复杂度	$O(\log n)$
最优时间复杂度	$O(1)$
空间复杂度	迭代：$O(1)$ 递归：$O(\log n)$

线性对数时间： O(nlogn)

$2^k \geq n \Rightarrow k \geq \log n$

- 归并排序、快速排序



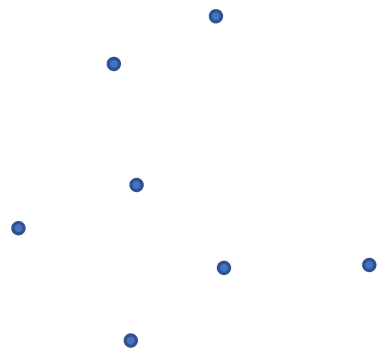
平方时间Quadratic: $O(n^2)$

- 插入排序

$$\sum_{i=2}^{n} i = (n+2)(n-1)/2$$

平方时间Quadratic: $O(n^2)$

- 平面点集的最短点对



```
min = dist(p1,p2)
for each point p:
    for each point q:
        d= dist(p,q)
        if d < dist:
            dist  = d
return min
```

立方时间Cubic: $O(n^3)$

- 矩阵的乘积：如2个n阶方阵相乘

```
for i=1 to n:
    for j=1 to n:
        cij = 0
        for k=1 to n:
            cij = cij+ aik*bkj
```

指数时间exponential: $O(a^n)$

- Hanoi汉诺塔问题： $O(2^n)$

```
Hanoi(n,A,B,C)
    if n==1:        move(n, A,C)
    Hanoi(n-1,A,C,B)
    move(n, A,C)
    Hanoi(n-1,B,C,A)
```

$T(n) = 2T(n-1)+1$

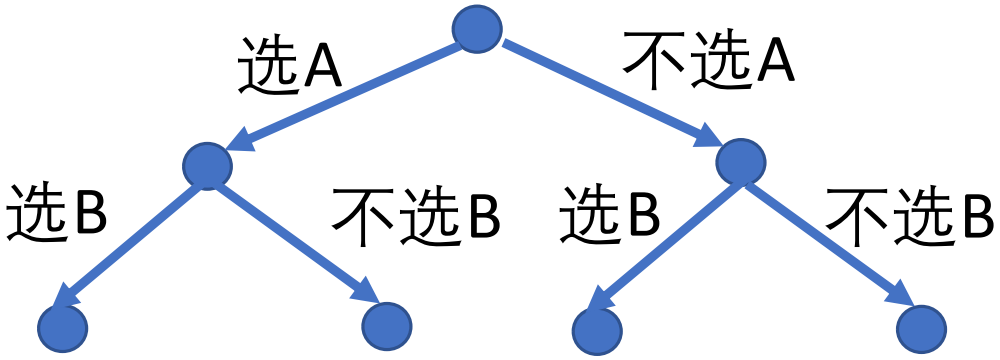
$T(1) = 1$

$\Rightarrow$ $T(n) = 2^n-1$

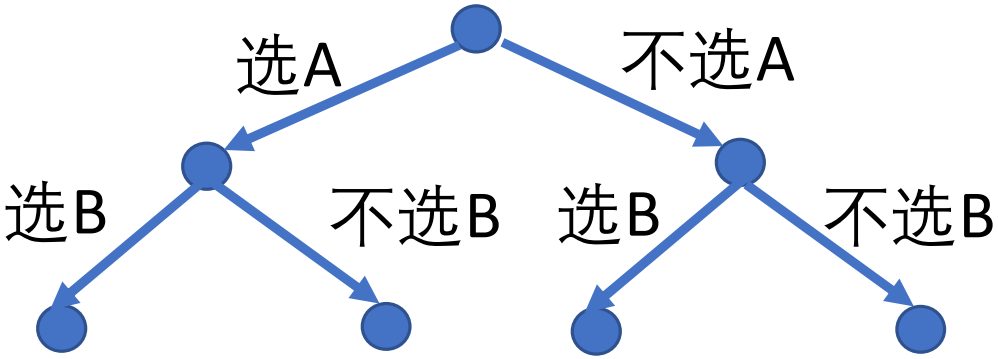
- **0/1背包问题**(Knapsack problem)

给定一组物品，每个物品有重量和价值，在背包容量有限的情况下，选择哪些物品放入背包，使得总重量不超过容量且总价值最大。每个物品只能选择放入或不放入背包。

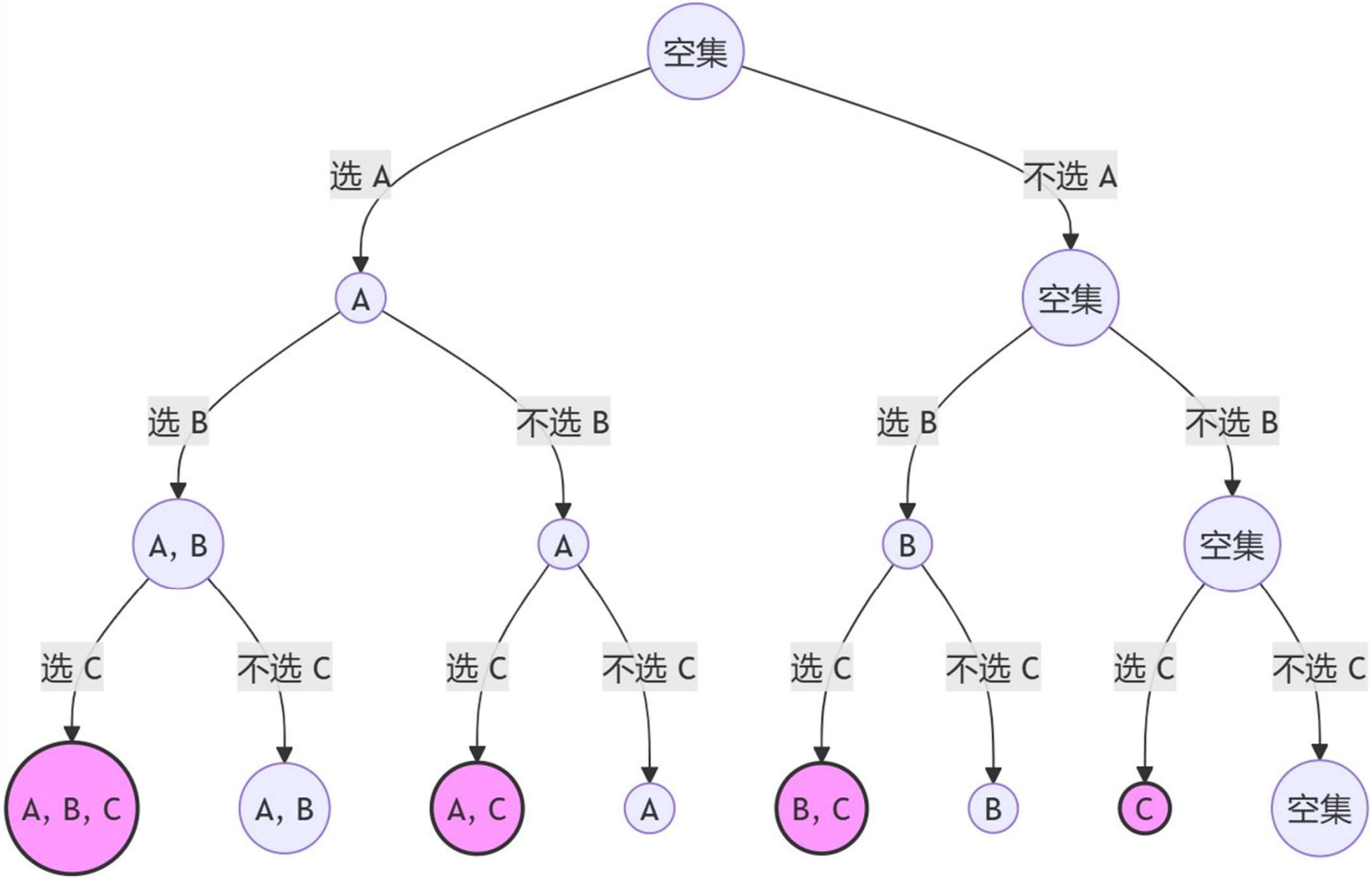
穷举法：通过枚举所有物品的选择组合，计算每种组合的总重量和总价值，选择符合容量限制且价值最大的组合。



- 穷举法： 每个物体选或不选，一共有2^n。$O(2^n)$



- 穷举法： 每个物体选或不选，一共有2^n。 $O(2^n)$



阶乘时间factorial：O(n!)

1. 全排列求解

给定一个包含n个元素的集合，生成该集合的所有排列。假设集合中的元素是不同的，可通过蛮力算法生成所有排列，数量是n!。算法的时间复杂度是O(n!)

例如，对于集合{1, 2, 3}，所有的排列是：。

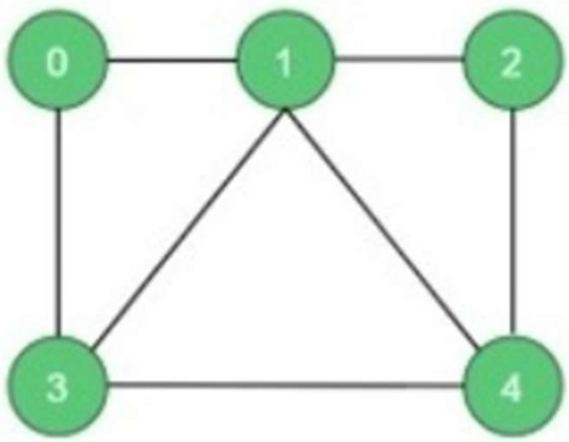
```
(1, 2, 3)
(1, 3, 2)
(2, 1, 3)
(2, 3, 1)
(3, 1, 2)
(3, 2, 1)
```

2.哈密尔顿回路问题（Hamiltonian Cycle Problem）

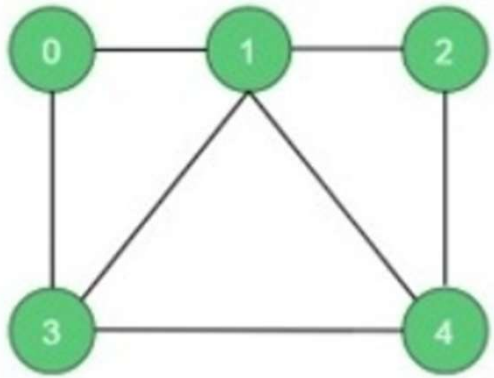
在给定的无向或有向图中，寻找一条经过所有顶点恰好一次并回到起点的闭合路径（回路）。

输入：一个图 G(V,E)，其中 V 是顶点集，E 是边集。

输出：是否存在哈密尔顿回路，如果存在，输出路径。



(0, 1, 2, 4, 3, 0)



蛮力法：

固定起点（比如 0 号顶点）。

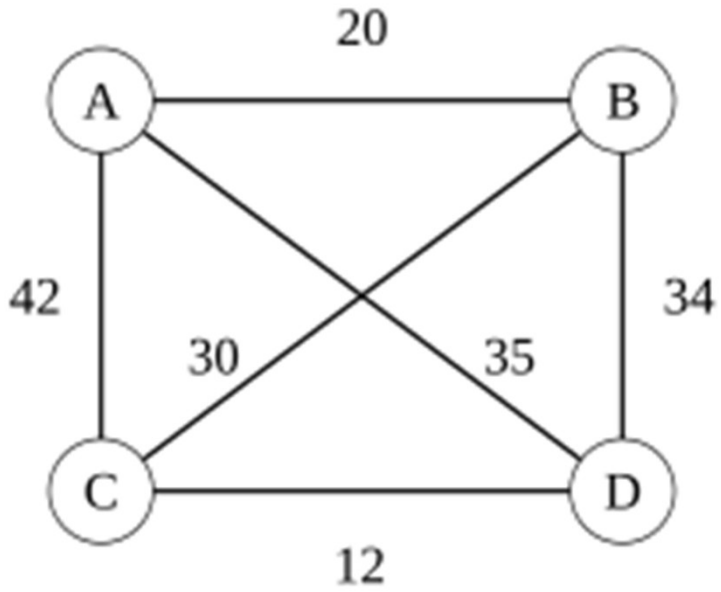
对剩余的n−1 个顶点进行全排列，共有 (n−1)! 种排列。

检查每个排列是否形成回路（即相邻顶点有边，并且最后一个顶点能回到起点）。n次判断

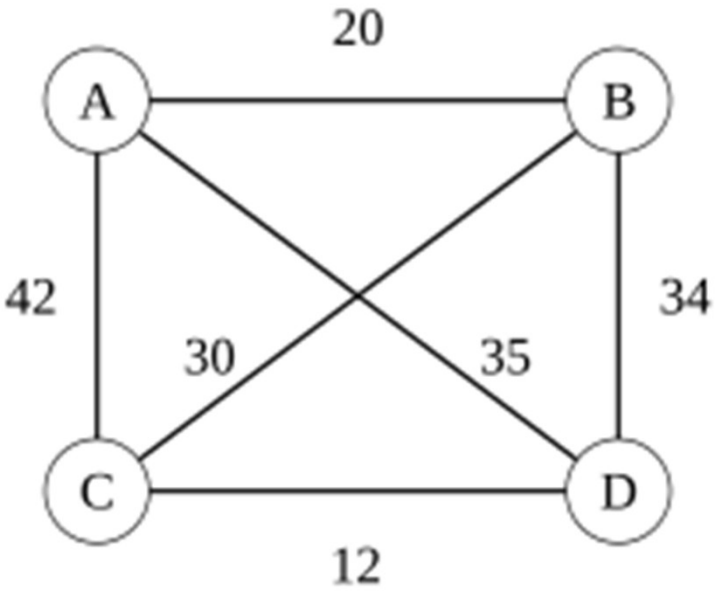
时间复杂度：O(n(n−1)!) = O(n!)

3. 旅行商问题（TSP，Travelling Salesman Problem）

在一个带权重的完全图G(V,E) 中，有n 个城市（顶点），每两座城市之间都有一条带有路径权重（距离、成本等）的边。目标是找到一条经过所有城市恰好一次并最终回到起点的最短路径。



- 通常采用蛮力法或动态规划（Held-Karp算法）求解。



蛮力法:

1. 枚举所有城市访问顺序（排列，共 $(n-1)!$ 种）。
2. 对每个排列:

    是否是构成哈密尔顿回路。//n次

    计算路径长度。 //n次加法

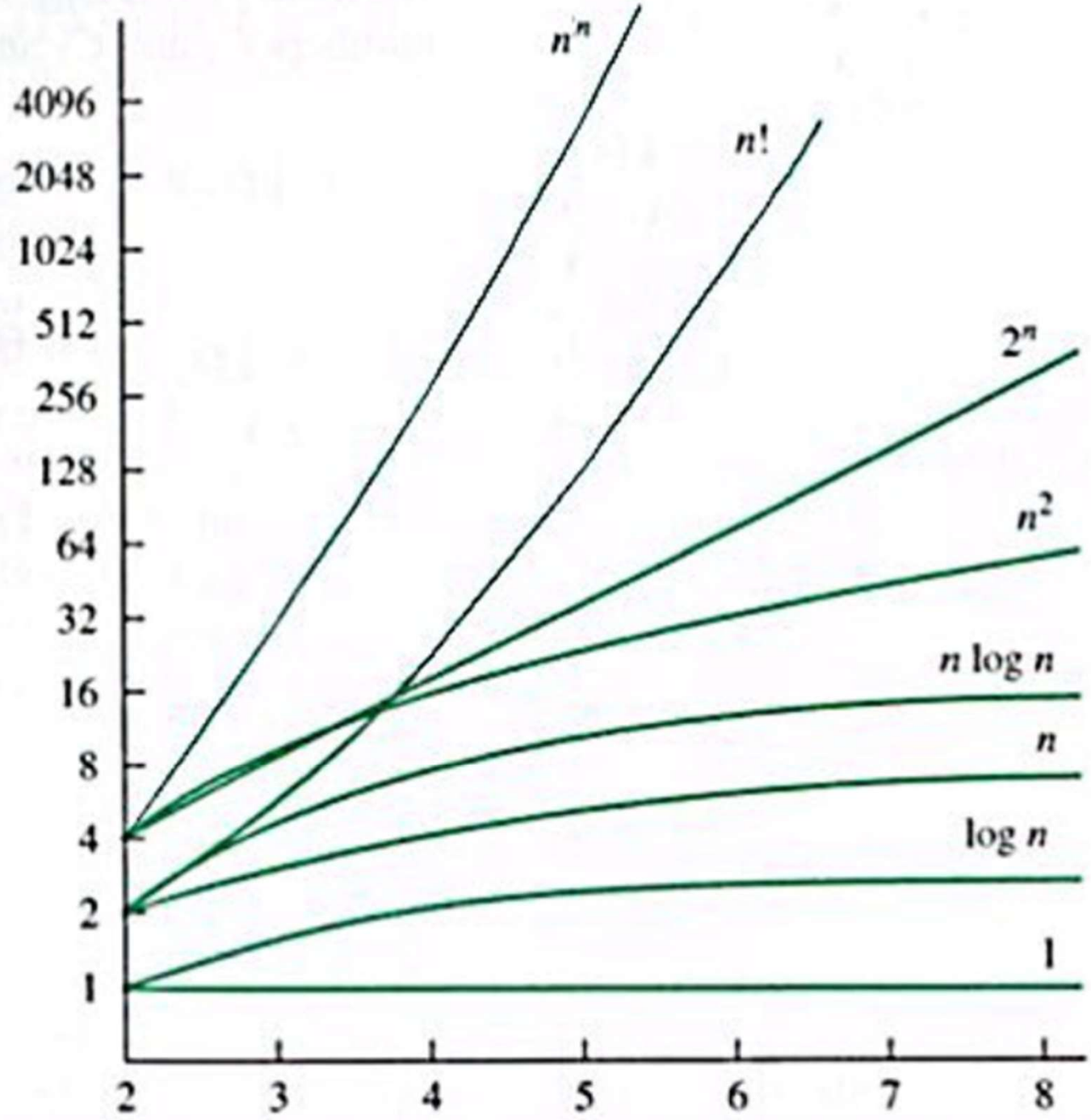
    记录最短路径。//1次比较、0到2次赋值

时间复杂度: $O(2n(n-1)!) = O(n!)$

常见时间复杂度类别

以下表格列出了算法分析中最常见的几种时间复杂度类别，按增长速度从慢到快排列：

记号	名称	增长趋势简述	典型场景示例
$\Theta(1)$	常数阶	时间固定，与 n 无关	数组下标访问、哈希表平均查找
$\Theta(\log n)$	对数阶	每次操作规模减半，增长极慢	二分查找、有序集合查找
$\Theta(n)$	线性阶	时间与 n 成正比	遍历数组、求最大值、线性查找
$\Theta(n \log n)$	线性对数阶	高效的分治算法常见	归并排序、快速排序平均情况、堆排序
$\Theta(n^2)$	平方阶	双重循环常见，n 较大时已明显变慢	冒泡排序、插入排序最坏情况、简单选择排序
$\Theta(2^n)$	指数阶	规模稍大即爆炸，通常只适用于小 n	汉诺塔、子集枚举、暴力解背包问题
$\Theta(n!)$	阶乘阶	增长最极端，仅限极小规模	全排列生成、旅行商问题（TSP）暴力解


n^n
$n!$
2^n
n^2
$n \log n$
n
$\log n$
1
4096
2048
1024
512
256
128
64
32
16
8
4
2
1
2
3
4
5
6
7
8

渐近分析的局限性

- **常数系数可能影响实际性能**

  两个算法可能具有相同的渐近复杂度（如），但其**常数系数**可能相差悬殊。例如，100n 和 2n 在n很大时都呈线性增长，但后者在同等条件下快 50 倍。因此，当渐近阶数相同时，必须考虑常数因子的影响。

渐近分析的局限性

- **低阶项在小规模输入时可能占主导**

示例:

- **算法 A（低阶但常数大）**：$f(n) = n + 100$ —— 复杂度 $O(n)$
- **算法 B（高阶但常数小）**：$f(n) = n^2$ —— 复杂度 $O(n^2)$

对比分析:

- 当 $n = 5$ 时：A 耗时 105，B 耗时 25。此时 $O(n^2)$ **的算法反而比** $O(n)$ **快。**
- 只有当 $n > 10.5$（跨越平衡点）后，$O(n)$ 的优势才会显现。

渐近分析的局限性

- **最坏情况与平均情况的差异**

　渐近分析通常默认讨论**最坏情况（Worst-case）**，但某些算法在实际运行中的**平均表现（Average-case）**远优于其最坏情况（如快速排序）。反之，有些算法（如归并排序）的最坏与平均情况高度一致。在进行算法选型时，应明确当前分析的是哪种场景。

渐近分析的局限性

- **隐藏的硬件与实现细节**

　理论分析中的“基本操作”在物理设备上的执行时间并不均等。现代计算机的内存访问速度、缓存命中率（Cache Hit）、分支预测等底层机制，可能导致理论复杂度相同的算法在实际运行中表现迥异。

常见函数

1）取整函数

$\lfloor x \rfloor$：不大于x的最大整数;

$\lceil x \rceil$：不小于x的最小整数。

2）指数函数

对于正整数m,n和实数a>0:

$a^0=1$;

$a^1=a$;

$a^{-1}=1/a$;

$(a^m)^n = a^{mn}$;

$(a^m)^n = (a^n)^m$;

$a^m a^n = a^{m+n}$;

$a>1 \Rightarrow$ $\Rightarrow n^b = o(a^n)$，即a^n是n^b的非紧上界

3）对数函数

$\log n = \log_2 n$;

$\lg n = \log_{10} n$;

$\ln n = \log_e n$;

$\log^k n = (\log n)^k$;

$\log\log n = \log(\log n)$;

for a>0,b>0,c>0

$$a = b^{\log_b a}$$

$$\log_c(ab) = \log_c a + \log_c b$$

$$\log_b a^n = n \log_b a$$

$$\log_b a = \frac{\log_c a}{\log_c b}$$

$$\log_b(1/a) = -\log_b a$$

$$\log_b a = \frac{1}{\log_a b}$$

$$a^{\log_b c} = c^{\log_b a} \qquad a^{\log_b n} = n^{\log_b a}$$

关注我

博客： hwdong-net.github.io

Youtube频道

hwdong

@hwdong · 5.01K subscribers · 558 videos

博客：https://hwdong-net.github.io >

youtube.com/c/4kRealSound and 4 more links